



Patterns Agentiques — Le Cours Illustré

Du Chatbot à l'Agent : donnez des mains, des yeux et un cerveau à votre IA

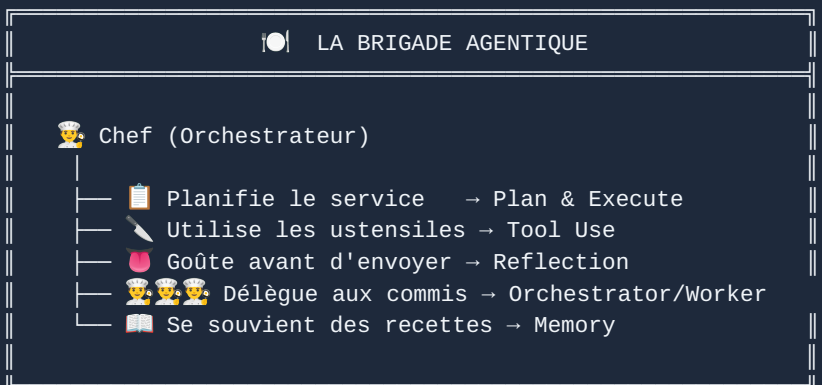


La Métaphore Fondatrice : La Cuisine d'un Grand Restaurant

Avant toute technique, ancrez cette image. Elle vous suivra tout le long.

Un chatbot, c'est un serveur qui vous récite le menu.

Un agent, c'est **toute la brigade en cuisine** : le chef planifie, les cuisiniers exécutent, on goûte avant d'envoyer, on se souvient des allergies du client, et on utilise les bons ustensiles.



Gardez cette cuisine en tête. Chaque pattern va y prendre vie.



Le Tableau de Bord — Vue Hélicoptère

Avant de plonger, voici la carte complète du territoire :

#	Pattern	La question qu'il résout	En cuisine...	Composant clé
1	Tool Use	Comment j'agis sur le monde ?	Le cuisinier prend un couteau	API / fonctions
2	Reflection	Comment je m'auto-corrige ?	On goûte la sauce	Boucle générer→évaluer
3	Plan & Execute	Comment je découpe une tâche complexe ?	Le chef écrit la fiche technique	Planificateur
4	Orchestrator / Worker	Comment je délègue à l'échelle ?	Le chef dispatche aux commis	Agent maître + workers
5	Memory & Context	Comment je ne sature pas le contexte ?	Le recipe book vs la mémoire du chef	Working + long-term memory

💡 **Le fil rouge** : un chatbot *répond*. Un agent **agit** → **vérifie** → **planifie** → **délègue** → **se souvient**. Ces 5 patterns sont les 5 organes vitaux d'un agent de production.

🔪 PATTERN 1 — Tool Use / Function Calling

« *Donner des mains au cerveau* »

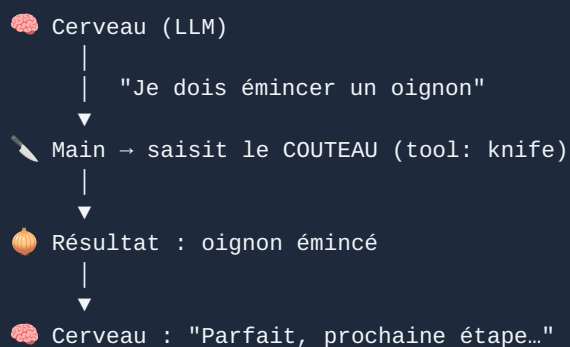
L'idée en une phrase :

*Un LLM seul est un cerveau dans un bocal : il pense brillamment mais ne peut rien toucher. Le **Tool Use**, ce sont les mains qu'on lui greffe.*

🔍 L'analogie cuisine :

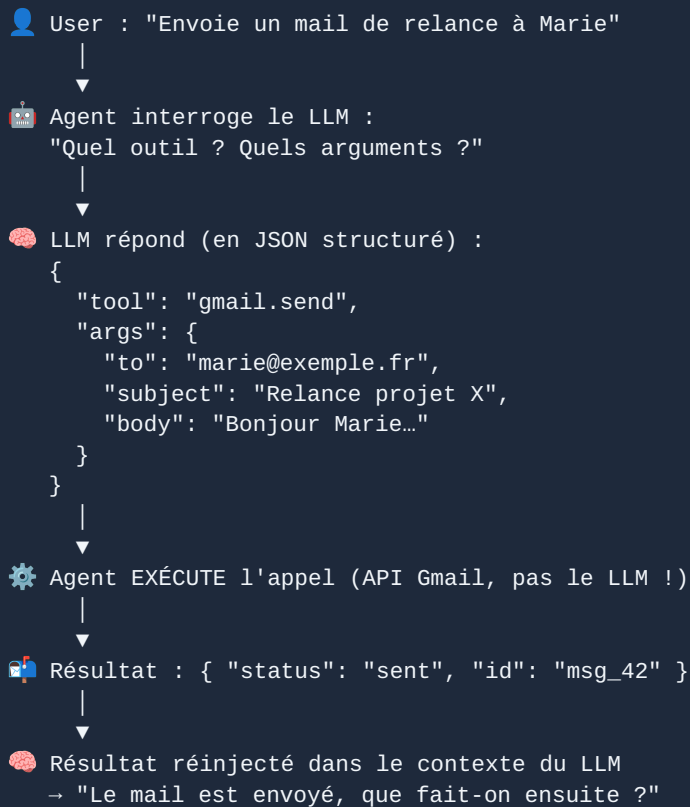
Imaginez un chef brillant... **ligoté à une chaise**. Il peut vous décrire parfaitement comment découper un oignon, mais sans couteau, sans planche, sans poêle — il ne peut rien faire.

Le Tool Use, c'est lui **déliier les mains** et lui ouvrir l'armoire à ustensiles :



Le mécanisme pas à pas :

CYCLE D'UN TOOL CALL



Les 4 étapes à retenir :

Étape	Qui fait quoi ?	🔥 En cuisine
1	L'utilisateur exprime un besoin	Le client commande
2	Le LLM choisit l'outil + arguments	Le chef dit "prends le wok, feu vif"
3	L'agent exécute l'appel réel	Le cuisinier utilise le wok
4	Le résultat revient au LLM	Le plat est prêt, le chef vérifie

⚠️ LE PIÈGE MORTEL : Des outils mal décrits = des appels catastrophiques.

Imaginez que vous donniez à un cuisinier un ustensile étiqueté simplement *"truc"*. Il ne saura pas quand l'utiliser, avec quels paramètres, ni ce qu'il retourne.

Règle d'or : le schéma de chaque outil doit être aussi strict qu'une signature de fonction typée.

❌ MAUVAIS :

```
{
  "name": "send",
  "description": "envoie quelque chose"
}
```

✅ BON :

```
{
  "name": "gmail_send_email",
  "description": "Envoie un email via l'API Gmail. Retourne l'ID du message si succès, une erreur SMTP sinon.",
  "parameters": {
    "to": { "type": "string", "format": "email", "required": true },
    "subject": { "type": "string", "maxLength": 200, "required": true },
    "body": { "type": "string", "required": true }
  }
}
```

🔧 Mini-exercice mental :

Vous construisez un agent qui gère un agenda. Décrivez l'outil `calendar_create_event` avec un schéma strict. Quels champs ? Quels types ? Quelle description ?

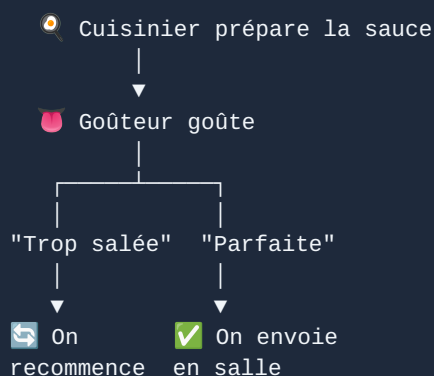
🍷 PATTERN 2 — Reflection

« Goûter avant d'envoyer en salle »

L'idée en une phrase :

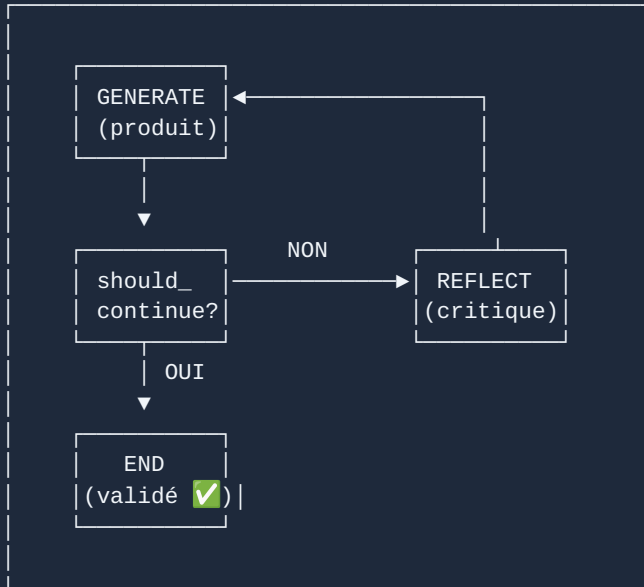
*Un agent qui produit sans vérifier, c'est un cuisinier qui envoie une sauce sans l'avoir goûtée. La **Reflection** ajoute un évaluateur qui dit : "Non, c'est trop salé, on recommence."*

🔍 L'analogie cuisine :



Le point crucial : **le goûteur n'est pas le cuisinier**. S'il se goûte lui-même, il sera toujours indulgent.

La boucle Generate → Evaluate :



Les 3 rôles :

Rôle	Fonction	🔍 En cuisine
Générateur	Produit une première sortie	Le cuisinier prépare le plat
Évaluateur	Score la sortie vs des critères	Le chef goûte et note
Boucle	Réitère jusqu'à condition d'arrêt	On recommence jusqu'à ce que ce soit bon

⚠️ **LE PIÈGE MORTEL : Une condition d'arrêt floue = une boucle infinie OU une sortie prématurée.**

❌ **MAUVAIS** : "Est-ce que c'est bon ?" → Le modèle dit toujours "oui"

✅ **BON** : "Le code compile-t-il ET les tests passent-ils ?" → Assertion binaire, vérifiable, extérieure au modèle

Règle d'or : la condition d'arrêt doit être une **assertion vérifiable** (exit code, test unitaire, validation de schéma), **jamais** une auto-évaluation du générateur par lui-même.

🎯 C'est exactement la discipline RED/GREEN : le gate bash est la fonction `should_continue`. Pas le jugement du modèle. Le bash dit "ça compile ou pas". Point.

🔧 Mini-exercice mental :

Vous avez un agent qui génère du code Python. Quelle serait votre condition d'arrêt ?

Réponse attendue : `pytest tests/ --tb=short` retourne exit code 0.

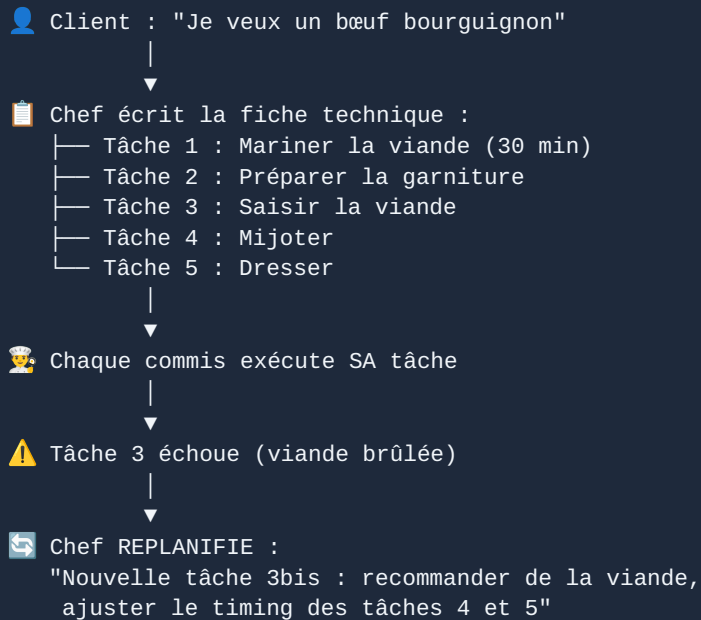
📋 PATTERN 3 — Plan & Execute

« *La fiche technique avant le coup de feu* »

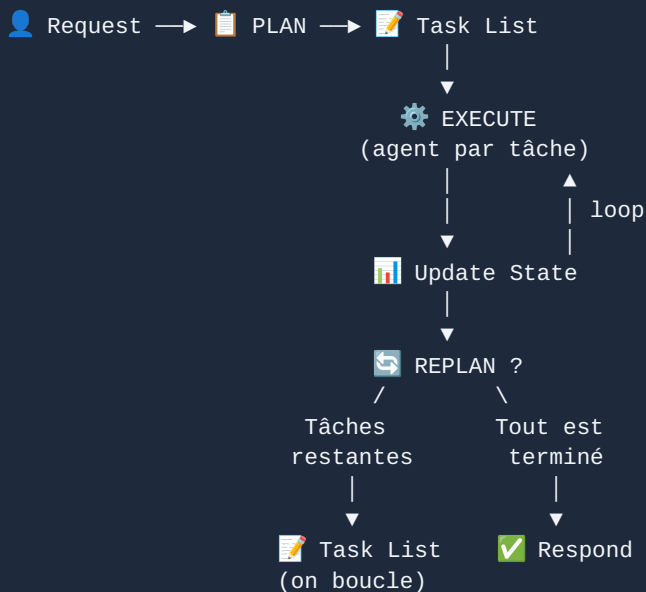
L'idée en une phrase :

Un chef ne dit pas "cuisinez un bœuf bourguignon" en un seul bloc. Il écrit une **fiche technique** : étape 1, étape 2, étape 3... et **réajuste** si la sauce accroche.

🔍 L'analogie cuisine :



Le flux complet :



Les 5 étapes :

Étape	Détail	🔍 En cuisine
1	Requête utilisateur brute	Le client commande
2	LLM génère une liste de sous-tâches ordonnées	Le chef écrit la fiche technique
3	Chaque tâche → agent dédié	Chaque commis fait sa partie
4	État mis à jour à chaque résultat	Le chef coche les étapes
5	Replanification : répondre OU reprendre	Ajustement en temps réel

⚠️ LE PIÈGE MORTEL : Un plan figé = un pipeline qui casse au premier imprévu.

Sans l'étape 5 (replan), un échec sur une sous-tâche **bloque tout**. C'est comme un chef qui refuse de changer la recette quand il n'y a plus de crème.

❌ **Plan rigide** : T1 → T2 → T3 (échec) → 💀 TOUT S'ARRÊTE

✅ **Plan vivant** : T1 → T2 → T3 (échec) → 🔄 Replan → T3bis → T4 → ✅

🔧 Mini-exercice mental :

Votre agent doit "Déployer une API en production". Listez 4 sous-tâches. Puis : que se passe-t-il si la tâche 2 (tests) échoue ? Quel est votre replan ?

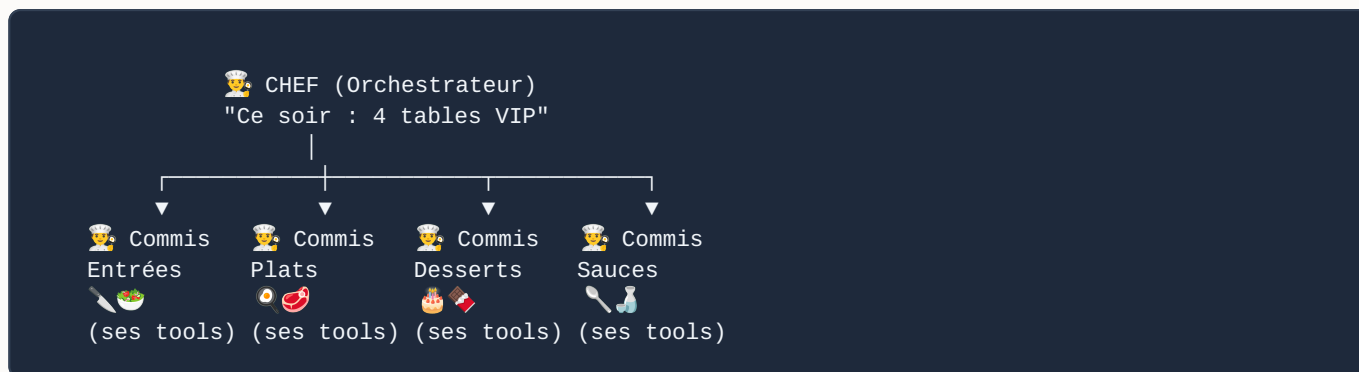
🎵 PATTERN 4 — Orchestrator / Worker

« *Le chef ne cuisine pas tous les plats* »

L'idée en une phrase :

Le chef d'un grand restaurant ne fait PAS tous les plats. Il **voit l'ensemble**, **distribue** les tâches, et **assemble** le résultat. Chaque commis a son propre poste, ses propres outils, son propre contexte limité.

🔍 L'analogie cuisine :

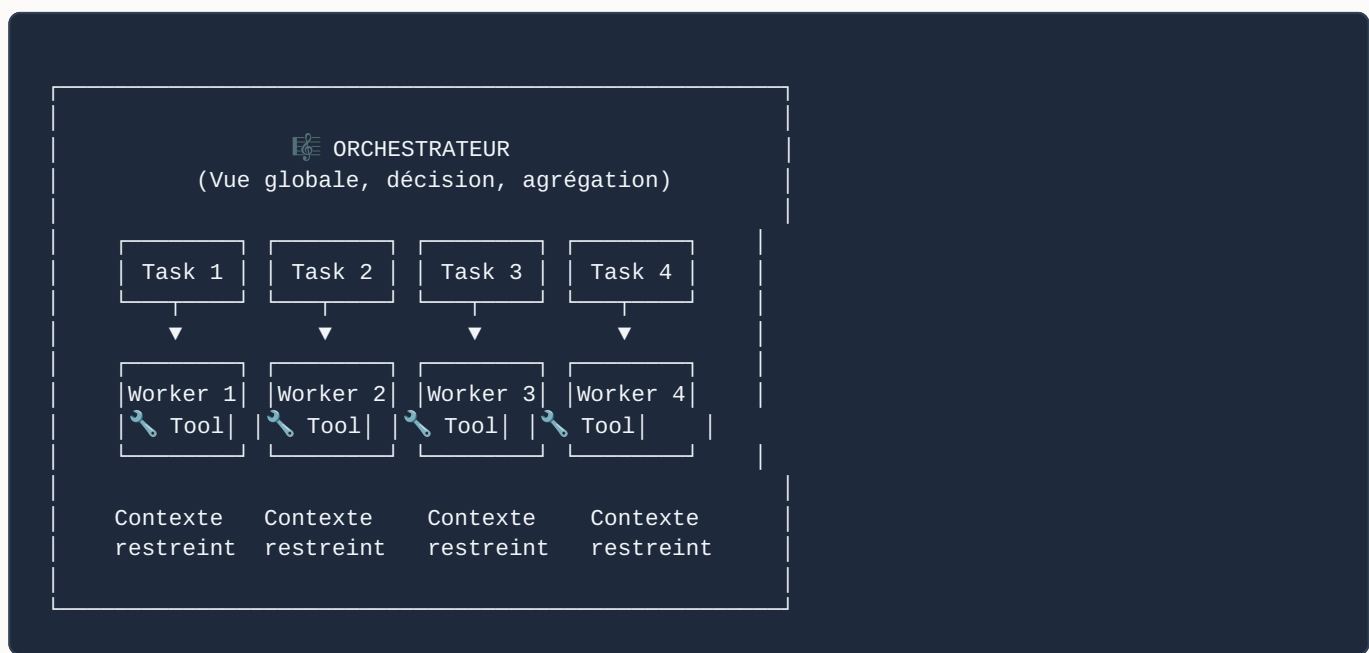


Chaque commis :

- Ne voit QUE sa tâche (contexte restreint)
- A SES propres ustensiles

- Ne connaît pas les commandes des autres tables
- Rapporte son résultat au chef

La structure :



Les rôles :

Rôle	Contexte	Responsabilité	👨‍🍳 En cuisine
Orchestrateur	Vue globale	Décompose, délègue, agrège	Le chef expédie
Worker	Contexte restreint	Exécute UNE tâche avec SES outils	Le commis à son poste

💡 **Le bénéfice massif : Contexte restreint par worker = moins de pollution = meilleur taux de réussite.**
 Imaginez que chaque commis doive connaître TOUTES les commandes de TOUTES les tables. Il serait submergé, ferait des erreurs. En limitant son contexte à SA tâche, il excelle.

⚠️ **LE PIÈGE MORTEL : Un orchestrateur qui micro-gère = on retombe dans la pollution de contexte.**
 ❌ **MAUVAIS** : L'orchestrateur envoie TOUT le contexte à chaque worker → Chaque worker est noyé sous l'information → On a juste créé un goulot d'étranglement déguisé.
 ✅ **BON** : L'orchestrateur envoie UNIQUEMENT la tâche précise, les paramètres nécessaires, et le format de sortie attendu.

Règle d'or de la résilience : un worker mort ne doit JAMAIS bloquer l'orchestrateur. Prévoyez des **chaînes de repli** (Worker 3 🧟 ? → Worker 3-bis prend le relais).

🔧 **Mini-exercice mental :**
 Votre orchestrateur délègue 5 tâches à 5 workers. Le worker 2 timeout. Que se passe-t-il avec et sans chaîne de repli ?

📖 PATTERN 5 — Memory & Context Management

« Le recipe book du chef »

L'idée en une phrase :

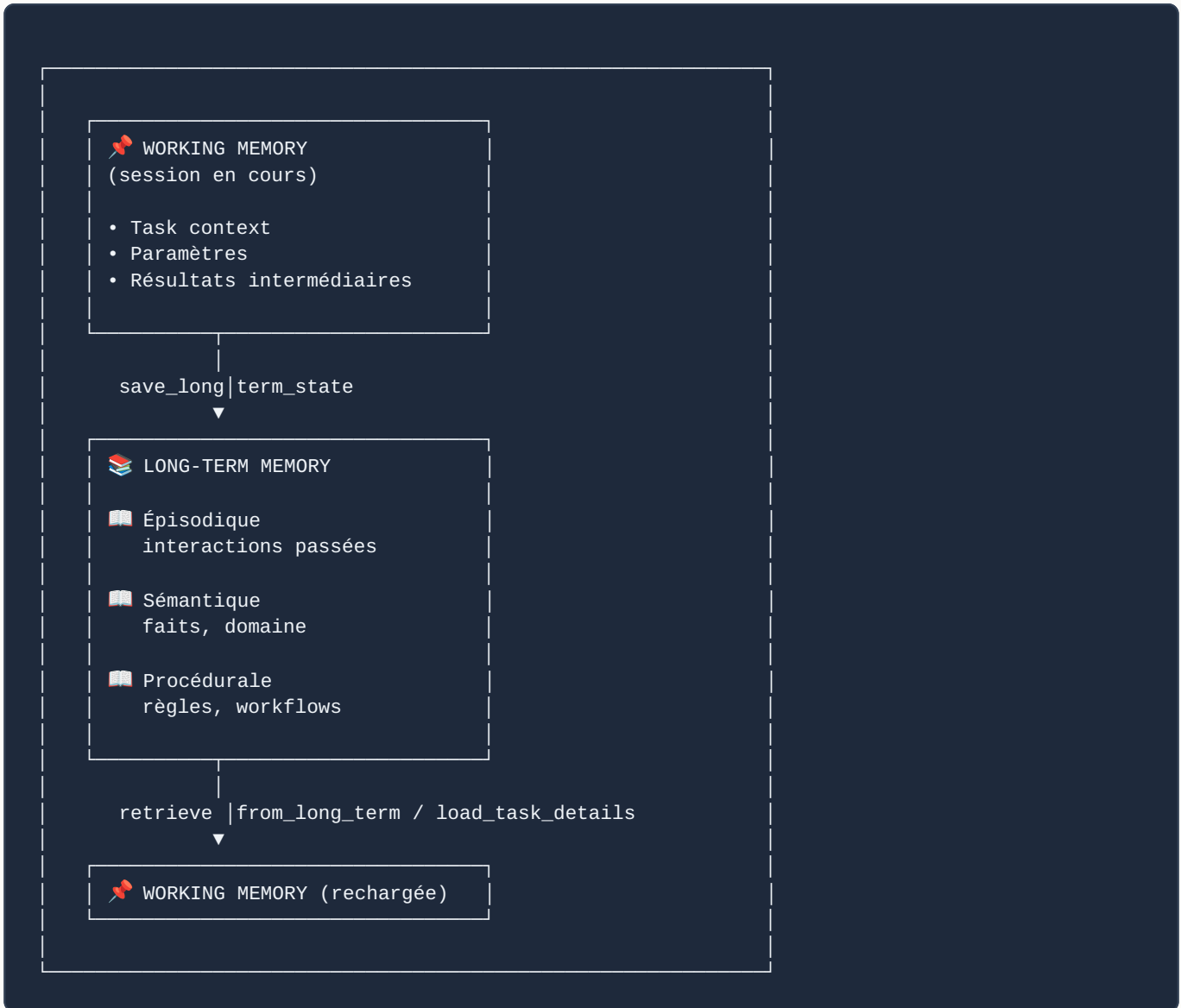
Un chef ne garde pas TOUT en tête. Il a sa **mémoire de travail** (la commande en cours) et son **recipe book** (années d'expérience). L'art est de **ne charger que ce qui est pertinent, au bon moment**.

🔍 L'analogie cuisine :

🧠 MÉMOIRE DU CHEF

- 📌 Working Memory (le service EN COURS)
"Table 7 : bœuf bourguignon, allergie noix, cuisson saignante, ticket depuis 12 min"
- 📖 Long-Term Memory (l'expérience)
 - 📖 Épisodique
"La table 7 revient chaque mardi, ils aiment le vin rouge"
 - 📖 Sémantique
"Le bœuf bourguignon nécessite 3h de cuisson, la farine sert à lier la sauce"
 - 📖 Procédurale
"Quand un client est allergique aux noix, je vérifie TOUJOURS la garniture ET la sauce"

L'architecture mémoire :



Les 4 types de mémoire :

Type	Contenu	🔍 En cuisine	Exemple tech
Working	État de la tâche en session	Le ticket en cours	Tool calls actifs, params
Épisodique	Interactions passées	"Ce client préfère la cuisson à point"	Décisions passées de l'user
Sémantique	Faits du domaine	"Le beurre noisette = 150°C"	Contraintes produit, préférences
Procédurale	Workflows appris	"Pour une allergie : vérifier sauce ET garniture"	Comment traiter tel type de demande

⚠️ **LE PIÈGE MORTEL** : Tout sauvegarder en long-term memory = fenêtre de contexte saturée dès le retrieve.

C'est comme un chef qui noterait CHAQUE commande, CHAQUE ingrédient utilisé, CHAQUE conversation depuis 10 ans dans un seul carnet. Au moment de chercher une info, il serait noyé.

❌ **MAUVAIS** : Tout écrire en LTM → retrieve → contexte explosé → 🧠💥

✓ **BON : FILTRER** à l'écriture : "Est-ce que cette info sera utile dans une session future ?"

Règle d'or : le filtre au moment de l'**écriture** est aussi important que le mécanisme de **récupération**.

 Mini-exercice mental :

Votre agent termine une tâche : "J'ai converti le fichier CSV en JSON".

- Est-ce utile en long-term memory ? (*Non, c'est éphémère*)
- Par contre : "L'utilisateur préfère l'encodage UTF-8 avec BOM". Utile ? (*Oui, sémantique*)

LA SYNTHÈSE — Check-list de l'Ingénieur IA

Voici votre **grille de lecture** à appliquer à tout projet agentique :

✓ CHECKLIST PRODUCTION AGENTIQUE

1. TOOL USE

Mes outils ont-ils un schéma d'arguments STRICT ?

☐ Oui, typé et documenté ☐ Non → à corriger

2. REFLECTION

Ma condition d'arrêt est-elle une ASSERTION VÉRIFIABLE ou un jugement du modèle ?

☐ Assertion (exit code, test) ☐ Jugement → danger

3. PLAN & EXECUTE

Est-ce que je REPLANIFIE après un échec, ou le pipeline casse ?

☐ Replan prévu ☐ Plan rigide → fragile

4. ORCHESTRATOR / WORKER

Un worker MORT bloque-t-il l'orchestrateur ?

☐ Chaîne de repli prévue ☐ Worker unique → SPOF

5. MEMORY

Est-ce que je FILTRE avant d'écrire en long-term, ou j'accumule tout ?

☐ Filtre à l'écriture ☐ Tout stocke → saturation

Le Test Final — Reconnaissez le Pattern

Pour chaque situation, identifiez le pattern en jeu :

Situation	Pattern ?
Votre agent appelle une API REST pour créer un ticket Jira	Tool Use
Votre agent génère du code, lance <code>pytest</code> , et recommence si ça échoue	Reflection

Situation	Pattern ?
Votre agent décompose "Refondre le site web" en 12 sous-tâches	Plan & Execute
Votre agent maître dispatche 3 tâches à 3 instances LLM différentes	Orchestrator / Worker
Votre agent se souvient que l'utilisateur préfère TypeScript à JavaScript	Memory (sémantique)

Le Mot de la Fin

Un chatbot RÉPOND.

Un agent : 🧠 **AGIT** (Tool Use) | 🍷 **VÉRIFIE** (Reflection) | 📋 **PLANIFIE** (Plan & Execute) | 🎵 **DÉLÈGUE**
(Orchestrator/Worker) | 🧠 **SE SOUVIENT** (Memory)

Ces 5 patterns ne sont pas des options. Ce sont les 5 organes vitaux de tout agent en production.

Vous avez maintenant la carte. Il ne reste plus qu'à cuisiner. 🔍